Structured Programming and CRR Code Lab

FRE6883 – Recitation 2

Complete it, trace it, test it, explain it

Today we rebuild one pricer from small testable pieces

Model data S_0, U, D, R, N, K

$\downarrow$

Stock-price nodes

$\downarrow$

Terminal payoffs

$\downarrow$

Backward induction

$\downarrow$

Option price $H(0,0)$

Your job on every activity

Predict the missing operation, complete the code, trace one example, and name one test that could expose a bug.

The goal is not to memorize the final function. It is to make every line explainable.

01 | Compute one stock-price node – Problem

GUIDED CODE Formula to function — Complete together

At step n and node i,

$$S(n,i) = S_0 U^i D^{n-i}, \qquad 0 \le i \le n.$$

```
double CalculateAssetPrice(
    double S0, double U, double D, int n, int i)
{
    return /* complete the expression */;
}
```

Hand check

$S_0 = 106$, $U = 1.15125$, $D = 0.86862$, $n = 3$, $i = 2$ should give approximately 122.03.

Think first: Which exponent counts up moves? Which counts down moves?

01 | Compute one stock-price node – Solution

```
double CalculateAssetPrice(
    double S0, double U, double D, int n, int i)
{
    return S0 * std::pow(U, i)
              * std::pow(D, n - i);
}
```

The idea

i counts up moves; $n - i$ counts down moves. Their sum is always n.

Boundary nodes

$i = 0$ gives $S_0 D^n$; $i = n$ gives $S_0 U^n$.

Required header

`std::pow` is declared in `<cmath>`.

02 | Visit every node exactly once – Problem

GUIDED CODE Nested loops — Complete the bounds

For $N = 3$, the valid node pairs are

$$(0,0);\ (1,0),(1,1);\ (2,0),(2,1),(2,2);\ (3,0),(3,1),(3,2),(3,3).$$

```
for (int n = /* start */; n /* condition */ N; ++n)
{
    for (int i = /* start */; i /* condition */ n; ++i)
    {
        std::cout << CalculateAssetPrice(
            S0, U, D, n, i) << std::endl;
    }
}
```

Think first: How many nodes should the code print when $N = 3$?

02 | Visit every node exactly once – Solution

```
for (int n = 0; n <= N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        std::cout << CalculateAssetPrice(
            S0, U, D, n, i) << std::endl;
    }
}
```

Outer loop

Selects one time step from 0 through N.

Inner loop

Visits its $n + 1$ valid nodes, from 0 through n.

Node count

$1 + 2 + 3 + 4 = 10$ when $N = 3$; in general $(N + 1)(N + 2)/2$.

03 | Store the visited nodes in one array – Problem

GUIDED CODE Array + running index

Complete three blanks

```
const int MAX_N = 8;
const int SIZE = (MAX_N + 1) * (MAX_N + 2) / 2;
double prices[SIZE] = { 0.0 };

int index = /* ? */;
for (int n = 0; n <= MAX_N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        prices[index] = CalculateAssetPrice(
            S0, U, D, n, i);
        /* advance index */;
    }
}
```

Think first: What is the first valid array index? What is the last?

03 | Store the visited nodes in one array – Solution

```
const int MAX_N = 8;
const int SIZE = (MAX_N + 1)
               * (MAX_N + 2) / 2;
double prices[SIZE] = { 0.0 };

int index = 0;
for (int n = 0; n <= MAX_N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        prices[index] = CalculateAssetPrice(
            S0, U, D, n, i);
        ++index;
    }
}
```

Capacity

Steps 0 through 8 contain $9 \cdot 10/2 = 45$ nodes.

Invariant

Before each write, `index` identifies the next free slot.

Safety

The final write uses index 44; then `index` becomes 45.

A loop generates nodes; an array remembers them

Computation

`CalculateAssetPrice` translates one mathematical rule into one reusable operation.

Traversal

The nested loops generate every valid pair (n, i).

Storage

The array preserves results after the loop moves to the next node.

Check the design

Can we read `prices[SIZE]`? Why or why not?

04 | Reject invalid model data – Problem

GUIDED CODE Conditions + early return

Complete the no-arbitrage test

Valid data must satisfy $S_0 > 0$ and $0 < D < R < U$.

```
int ValidateInputData(
    const double& S0, const double& U,
    const double& D, const double& R)
{
    if (S0 <= 0.0 || D <= 0.0 || U <= D)
    {
        return -1;
    }
    if (/* reject R outside (D,U) */)
    {
        return -1;
    }
    return 0;
}
```

Test: should $D = 0.9$, $R = 0.9$, $U = 1.1$ pass?

04 | Reject invalid model data – Solution

```
int ValidateInputData(
    const double& S0, const double& U,
    const double& D, const double& R)
{
    if (S0 <= 0.0 || D <= 0.0 || U <= D)
    {
        return -1;
    }
    if (R <= D || R >= U)
    {
        return -1;
    }
    return 0;
}
```

Boundary matters

Equality creates an invalid boundary; use <= and >=.

Useful test

With $D = R = 0.9$, the function returns −1.

Why early return

Invalid inputs cannot reach the pricing calculation.

Functions give each financial rule one clear home

BinomialTreeModel.h

```
#pragma once
namespace fre {
double RiskNeutProb(
    double U, double D, double R);
double CalculateAssetPrice(
    double S0, double U, double D,
    int n, int i);
}
```

Declarations tell other files what exists.

BinomialTreeModel.cpp

```
#include "BinomialTreeModel.h"
namespace fre {
// definitions belong here
}
```

Definitions contain the executable rules.

Design question: which file should include `<cmath>`?

05 | Implement the risk-neutral weight – Problem

GUIDED CODE Small pure function — Complete and hand-check

$$q = \frac{R - D}{U - D}.$$

```
namespace fre {

double RiskNeutProb(double U, double D, double R)
{
    return /* ? */;
}

}
```

Hand check

For $U = 1.2$, $D = 0.8$, $R = 1.1$, the result should be 0.75.

Think first: Which validation condition guarantees that $0 < q < 1$?

05 | Implement the risk-neutral weight – Solution

```
namespace fre {

double RiskNeutProb(double U, double D, double R)
{
    return (R - D) / (U - D);
}

}
```

Parentheses

They preserve the numerator and denominator from the formula.

Test value

$q = (1.1 - 0.8)/(1.2 - 0.8) = 0.75$.

Precondition

$D < R < U$ places $R - D$ strictly between 0 and $U - D$.

06 | Predict value and reference effects – Problem

GUIDED CODE Trace before running — Fill the final output

```
void Foo(int a, int& b)
{
    ++a;
    ++b;
    std::cout << a << " " << b << std::endl;
}

int main()
{
    int x = 1, y = 1;
    Foo(x, y);
    std::cout << x << " " << y << std::endl;
}
```

Complete the trace

Inside `Foo`: __ __ Back in `main`: __ __

06 | Predict value and reference effects – Solution

Observed output

Inside `Foo`: `2 2`

Back in `main`: `1 2`

`int a`

The function receives a copy of `x`. Incrementing the copy does not change `x`.

`int& b`

The function receives an alias for `y`. Incrementing `b` changes `y`.

`const int& b`

The alias could be read but not incremented; `++b` would fail to compile.

07 | Return four inputs through references – Problem

GUIDED CODE Output parameters

Complete the reads and success code

```
int GetInputData(
    double& S0, double& U, double& D, double& R)
{
    std::cout << "Enter S0 U D R: ";
    std::cin >> /* four output parameters */;

    if (ValidateInputData(S0, U, D, R) == -1)
    {
        return /* failure */;
    }
    return /* success */;
}
```

Call site

```
double s0=0.0, u=0.0, d=0.0, r=0.0;
int status = fre::GetInputData(s0, u, d, r);
```

07 | Return four inputs through references – Solution

```
int GetInputData(
    double& S0, double& U, double& D, double& R)
{
    std::cout << "Enter S0 U D R: ";
    std::cin >> S0 >> U >> D >> R;

    if (ValidateInputData(S0, U, D, R) == -1)
    {
        return -1;
    }
    return 0;
}
```

Why references

One function call updates four variables owned by the caller.

Meaningful status

0 means success; −1 means invalid model data.

Caller responsibility

Check the status before attempting any pricing.

Each Lecture 2 function enters CRR at a specific moment

Before pricing

`GetInputData` reads the model inputs. `ValidateInputData` accepts or rejects them.

`OptionPricer01.cpp` calls `PriceByCRR` only after valid data.

Inside `PriceByCRR`

`RiskNeutProb` supplies q. At expiry, `CalculateAssetPrice` supplies $S(N, i)$ and `CallPayoff` supplies $H(N, i)$. Backward induction then combines child values.

08 | Implement a European call payoff – Problem

GUIDED CODE Conditional rule — Complete the function

$$h(z) = (z - K)^+ = \max(z - K, 0).$$

```
double CallPayoff(double z, double K)
{
    if (/* in the money */)
    {
        return /* positive payoff */;
    }
    return /* otherwise */;
}
```

Acceptance cases for $K = 100$

$z = 80 \to 0, \qquad z = 100 \to 0, \qquad z = 120 \to 20.$

Boundary question: Should the condition use > or >=?

08 | Implement a European call payoff – Solution

```
double CallPayoff(double z, double K)
{
    if (z > K)
    {
        return z - K;
    }
    return 0.0;
}
```

At the strike

$z - K = 0$, so both > and >= return zero. > states "positive payoff" directly.

One responsibility

The function knows the payoff rule, not how stock prices are generated.

Easy to test

No array or loop is required to verify the three cases.

09 | At expiry CRR calls two functions – Problem

GUIDED CODE Loop + two function calls — Complete the row

At expiry N, `PriceByCRR` first calls `CalculateAssetPrice` for $S(N, i)$, then calls `CallPayoff` to obtain $H(N, i) = h(S(N, i))$.

```
double Price[SIZE] = { 0.0 };

for (int i = /* first node */; i /* last node */ N; ++i)
{
    double stock = /* compute S(N,i) */;
    Price[i] = /* apply the payoff */;
}
```

Trace this case

$S_0 = 106$, $U = 1.15125$, $D = 0.86862$, $K = 100$, $N = 2$.

Complete the stock row [___, ___, ___] and payoff row [___, ___, ___].

09 | At expiry CRR calls two functions – Solution

```
double Price[SIZE] = { 0.0 };

for (int i = 0; i <= N; ++i)
{
    double stock = CalculateAssetPrice(
        S0, U, D, N, i);
    Price[i] = CallPayoff(stock, K);
}
```

Stock row

$[79.98, 106.00, 140.49]$ for $i = 0, 1, 2$.

Payoff row

$[0, 6.00, 40.49]$ for a call struck at 100.

Invariant

After the loop, indices 0 through N hold the complete expiry row.

After expiry, CRR needs q and two child values

`PriceByCRR` calls `RiskNeutProb(U, D, R)` once; it returns $q \approx 0.48413$. With $R = 1.00545$, begin with the terminal row $[0, 6.00, 40.49]$.

Step	Active values	Update rule
$n = 2$	$[0, 6.00, 40.49]$	terminal call payoffs
$n = 1$	$[2.8890, 22.5745]$	$H(1,i) = \frac{qH(2,i+1)+(1-q)H(2,i)}{R}$
$n = 0$	$[12.3521]$	same local rule

Why one array is enough

After computing row n, the old row $n+1$ is never needed again. We overwrite it from left to right.

10 | Complete backward induction – Problem

GUIDED CODE Two loops + in-place update — Complete four blanks

```
for (int n = /* first backward step */;
     n /* continue through root */ 0; --n)
{
    for (int i = 0; i /* nodes on row n */ n; ++i)
    {
        Price[i] =
            (q * /* up child */
             + (1.0 - q) * /* down child */) / R;
    }
}
return Price[0];
```

Index map

At node (n, i), the down child is `Price[i]` and the up child is `Price[i+1]`.

10 | Complete backward induction – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
return Price[0];
```

Outer loop

Expiry is already known, so begin at $N-1$ and finish at the root.

Inner loop

Row n contains indices 0 through n.

Final state

After $n = 0$, `Price[0]` is $H(0,0)$.

11 | Find the overwrite bug – Problem

GUIDED CODE Data dependency

The formulas are right; the result is wrong

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = n; i >= 0; --i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
```

Trace only row $n = 1$

Start from `Price = [0, 6.00, 40.49]`. Which entry is overwritten first? Which later calculation reads that changed entry as if it were still a child?

Repair: Change only the inner-loop header.

11 | Find the overwrite bug – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
```

What went wrong

With decreasing `i`, `Price[1]` changes before node $(1, 0)$ reads it as its up child.

Why increasing works

Updating `Price[0]` cannot damage `Price[1]`, which is still needed next.

General lesson

When overwriting an array, loop direction is part of correctness.

12 | PriceByCRR begins with validation and q – Problem A

GUIDED CODE Guard + initialization

Complete the preconditions

```
double PriceByCRR(double S0, double U, double D,
                  double R, const int N, double K)
{
    const int SIZE = 9;
    if (N < 1 || N >= SIZE || S0 <= 0.0 || K < 0.0
        || /* require 0 < D < R < U */)
    {
        std::cerr << "Invalid model data" << std::endl;
        return -1.0;
    }

    double q = /* call RiskNeutProb(U, D, R) */;
    double Price[SIZE] = { 0.0 };
    // Next: construct the terminal row.
```

Design check

Why does `SIZE = 9` allow $N = 8$ but reject $N = 9$?

12 | PriceByCRR builds expiry then moves backward – Problem B

GUIDED CODE Terminal row + backward pass — Connect the completed pieces

```
    for (int i = 0; i <= N; ++i)
    {
        Price[i] = /* payoff at S(N,i) */;
    }

    for (int n = /* ? */; n >= 0; --n)
    {
        for (int i = 0; i <= n; ++i)
        {
            Price[i] = /* one CRR update */;
        }
    }
    return /* root value */;
}
```

Before the reveal

Point to the base case, the recurrence, and the final answer in this function.

12 | PriceByCRR begins with validation and q – Solution A

```
double PriceByCRR(double S0, double U, double D,
                  double R, const int N, double K)
{
    const int SIZE = 9;
    if (N < 1 || N >= SIZE || S0 <= 0.0 || K < 0.0
        || D <= 0.0 || U <= D || R <= D || R >= U)
    {
        std::cerr << "Invalid model data" << std::endl;
        return -1.0;
    }

    double q = RiskNeutProb(U, D, R);
    double Price[SIZE] = { 0.0 };
    for (int i = 0; i <= N; ++i)
    {
        Price[i] = CallPayoff(
            CalculateAssetPrice(S0, U, D, N, i), K);
    }
```

The validation establishes safe ranges; the terminal loop establishes the base case.

12 | PriceByCRR builds expiry then moves backward – Solution B

```
    for (int n = N - 1; n >= 0; --n)
    {
        for (int i = 0; i <= n; ++i)
        {
            Price[i] =
                (q * Price[i + 1]
                 + (1.0 - q) * Price[i]) / R;
        }
    }
    return Price[0];
}
```

Shared example

$N = 8$ returns approximately 21.68 for $S_0 = 106$, $U = 1.15125$, $D = 0.86862$, $R = 1.00545$, $K = 100$.

Explain the memory

At any moment, the active prefix of `Price` represents one complete time level.

A CRR node gives both an option value and a delta hedge

At a node (n, i), replicate the option over one step with shares of stock and risk-free cash:

$$\Delta(n,i) = \frac{H(n+1,i+1) - H(n+1,i)}{S(n+1,i+1) - S(n+1,i)}.$$

Interpretation

Hold Δ shares. Put the remaining amount in the risk-free account (or borrow it when it is negative). The portfolio has the same up and down values as the option.

Where the stock values come from

`CalculateAssetPrice` provides the two child prices $S(n+1,i)$ and $S(n+1,i+1)$. The array `Price` holds the corresponding option values.

13 | Compute delta before overwriting `Price[i]`

```
double Su = CalculateAssetPrice(S0, U, D, n + 1, i + 1);
double Sd = CalculateAssetPrice(S0, U, D, n + 1, i);
double delta = (Price[i + 1] - Price[i]) / (Su - Sd);

Price[i] = (q * Price[i + 1]
            + (1.0 - q) * Price[i]) / R;
```

Lecture 2 $N = 2$ root

$S_d = 92.0737$, $S_u = 122.0325$, $H_d = 2.8890$, $H_u = 22.5745$.

Hedge

$\Delta \approx 0.6571$: hold 0.6571 share. Cash is $H(0,0) - \Delta S_0 \approx -57.30$, so borrow 57.30.

Order matters

Calculate delta before replacing `Price[i]`; it needs both child option values.

OptionPricer01.cpp runs the Lecture 2 example

```cpp
#include "BinomialTreeModel.h"
#include "Option01.h"
#include <iostream>
#include <iomanip>

int main()
{
    double u = 1.15125, d = 0.86862;
    double r = 1.00545, s0 = 106.00;
    double k = 100.00;
    const int N = 8;
    double optionPrice =
        fre::PriceByCRR(s0, u, d, r, N, k);
    std::cout << "European call option price = "
              << std::fixed << std::setprecision(2)
              << optionPrice << std::endl;
    return 0;
}
```

Expected output

```
European call option
price = 21.68
```

Same file roles

The model helpers live in `BinomialTreeModel`; the payoff and pricer live in `Option01`.

Tests should target boundaries and invariants

Case	Change	Expected observation	Bug targeted
Lecture example	$S_0 = 106, U = 1.15125, D = .86862, R = 1.00545, K = 100, N = 8$	21.68	full data flow
At strike	call payoff $z = K = 100$	0	payoff boundary
No-arbitrage edge	$R = D$	rejected	wrong inequality
Capacity edge	$N = 9$ with `SIZE=9`	rejected	out-of-bounds write
Zero strike	set $K = 0$	equal to $S_0 = 106$	discount/update error

A useful explanation has three parts

State the input, predict the result or invariant, then identify the line that would be suspect if the test failed.

One algorithm now has a line-by-line explanation

Validate inputs

↓

Compute q

↓

Build terminal payoffs

↓

Move backward in place

↓

Return `Price[0]`

Exit ticket

1. Why must R lie strictly between D and U?
2. Why does the inner CRR loop move from left to right?
3. Which function would you test first after changing the payoff from call to put?

If you can answer all three, you can debug the pricer rather than merely rerun it.

Additional practice reuses the same coding habits

For each problem

1. Trace the example by hand.
2. Complete the function.
3. Test one boundary case.
4. Explain the loop invariant.

Choose a useful next challenge

Count and accumulate first, then move to nested loops or a one-pass financial algorithm.

These problems come from Recitation 1 Deck 5 and can be used in class or after the core CRR lab.

Use `<vector>`; the last two solutions also use `<algorithm>`.

Practice 01 | Count positive values – Problem

GUIDED CODE Counter + condition Complete the function

Return how many elements of an integer vector are strictly positive. Zero does not count.

Examples

`[-2, 0, 5, 3, -1]` → 2 `[0, -4]` → 0

```
int countPositive(const std::vector<int>& nums)
{
    int count = /* ? */;
    for (int i = 0; i < static_cast<int>(nums.size()); ++i)
    {
        if (/* current value is positive */)
        {
            /* update count */;
        }
    }
    return /* ? */;
}
```

Practice 01 | Count positive values – Solution

```cpp
int countPositive(const std::vector<int>& nums)
{
    int count = 0;
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i)
    {
        if (nums[i] > 0)
        {
            ++count;
        }
    }
    return count;
}
```

Invariant

`count` equals the number of positive values already visited.

Boundary

An empty vector returns zero because the loop never runs.

Cost

$O(n)$ time and $O(1)$ extra space.

Practice 02 | Running sum – Problem

GUIDED CODE Accumulate a result — Complete the function

Return a vector whose value at index i is the sum of inputs from index 0 through i.

Example

`[2, 1, 3, 4]` → `[2, 3, 6, 10]`

```
std::vector<int> runningSum(const std::vector<int>& nums)
{
    int n = static_cast<int>(nums.size());
    std::vector<int> result(n);
    int total = /* ? */;
    for (int i = 0; i < n; ++i)
    {
        /* update total */;
        result[i] = /* ? */;
    }
    return result;
}
```

Practice 02 | Running sum – Solution

```cpp
std::vector<int> runningSum(
    const std::vector<int>& nums)
{
    int n = static_cast<int>(nums.size());
    std::vector<int> result(n);
    int total = 0;
    for (int i = 0; i < n; ++i)
    {
        total += nums[i];
        result[i] = total;
    }
    return result;
}
```

Invariant

After index i, `total` equals `nums[0]+...+nums[i]`.

Reuse

The next iteration adds one value instead of recomputing the prefix.

Cost

$O(n)$ time and $O(n)$ output space.

Practice 03 | Richest customer – Problem

GUIDED CODE Nested loops + maximum — Complete the resets and updates

Each row contains one customer's balances. Return the largest row total.

Example: `[[2, 3], [1, 8], [4, 4]]` → `9`

```
int maximumWealth(
    const std::vector<std::vector<int>>& accounts)
{
    int best = 0;
    for (int r = 0; r < static_cast<int>(accounts.size()); ++r)
    {
        int total = /* reset for this row */;
        for (int c = 0; c < static_cast<int>(accounts[r].size()); ++c)
        {
            /* add one balance */;
        }
        best = /* keep the larger value */;
    }
    return best;
}
```

Practice 03 | Richest customer – Solution

```cpp
int maximumWealth(
    const std::vector<std::vector<int>>& accounts)
{
    int best = 0;
    int rows = static_cast<int>(accounts.size());
    for (int r = 0; r < rows; ++r)
    {
        int total = 0;
        int banks = static_cast<int>(accounts[r].size());
        for (int c = 0; c < banks; ++c)
        {
            total += accounts[r][c];
        }
        best = std::max(best, total);
    }
    return best;
}
```

Inner loop

Computes one customer's total.

Outer loop

Compares that total with the best completed row.

Critical reset

`total = 0` belongs inside the outer loop.

Practice 04 | Best time to trade – Problem

GUIDED CODE Track the best value so far — Complete one pass

Return the maximum profit from one buy followed by one later sell. Return zero when no profitable trade exists. Assume the vector contains at least one price.

Examples: `[7,1,5,3,6,4]` → 5 `[7,6,4,3,1]` → 0

```
int maxProfit(const std::vector<int>& prices)
{
    int minPrice = prices[0];
    int best = 0;
    for (int i = 1; i < static_cast<int>(prices.size()); ++i)
    {
        int profit = /* sell today */;
        best = /* best profit so far */;
        minPrice = /* cheapest price so far */;
    }
    return best;
}
```

Practice 04 | Best time to trade – Solution

```cpp
int maxProfit(const std::vector<int>& prices)
{
    int n = static_cast<int>(prices.size());
    int minPrice = prices[0];
    int best = 0;
    for (int i = 1; i < n; ++i)
    {
        int profit = prices[i] - minPrice;
        best = std::max(best, profit);
        minPrice = std::min(minPrice, prices[i]);
    }
    return best;
}
```

Invariant

Before day i, `minPrice` is the cheapest earlier price.

Order matters

Evaluate today's sale before adding today's price to the past.

Cost

$O(n)$ time and $O(1)$ extra space.

The same habits transfer beyond CRR

State a loop invariant

Name what each accumulator, index, or active array prefix means before and after an iteration.

Protect boundaries

Test empty inputs, equality cases, first and last indices, and invalid model parameters.

Explain before optimizing

A correct explanation connects the specification, the stored state, the update, and a test that could falsify the code.

That is the common skill behind every exercise in this deck.